

会议室预约与时间冲突检查系统

软件概要设计以及详细设计

专 业:	软件工程
方 向:	软件开发与测试
项目名称:	会议室预约与时间冲突检查系统
队伍名称	尖草坪游击队
项目成员:	李宗熹、翟浩森、张陈浩、闫佳馥、 霍汶洺
辅导教师:	张淑娟

2026 年 9 月

目录

1 引言与设计依据	1
1.1 编写目的	1
1.2 设计依据	1
1.3 设计原则	2
1.4 术语与约定	3
2 系统总体设计	3
2.1 系统设计目标	3
2.2 技术路线	4
2.3 前后端交互结构	5
2.5 部署与运行结构	8
3 系统模块详细设计	10
3.1 模块总览	10
3.2 identity 身份与认证模块	12
3.3 resource 会议室资源模块	14
3.4 reservation 预约核心模块	15
3.5 meeting 会议执行模块	18
3.6 administration 管理与审计模块	19
3.7 前端模块设计	20
3.8 模块协作与集成约束	22
3.9 本章小结	24
4 数据库设计	24

4.1 数据库总体设计	24
4.2 主键、外键与关系设计	26
4.3 数据表字段设计	26
4.4 索引与约束设计	31
4.5 数据库并发一致性设计	32
5 接口设计	32
5.1 接口基本约定	32
5.2 当前已实现 API	33
5.3 规划 API	33
5.4 创建预约接口	34
5.5 错误码与 HTTP 状态语义	34
6 核心业务详细设计	35
6.1 创建预约程序流程	35
6.2 时间冲突判定算法	错误！未定义书签。
6.3 并发控制与事务边界	37
6.4 幂等设计	39
6.5 预约状态机	41
6.6 展示状态推导	43
6.7 取消预约详细设计	44
6.8 特殊会议室审批详细设计（规划）	44
6.9 前端冲突反馈与刷新	44

7 安全、异常与可维护性设计	45
7.1 身份认证与授权	46
7.2 统一异常处理	47
7.3 可维护性约束	47
8 部署与验证设计	47
8.1 开发运行环境	47
8.2 构建与测试	47
8.3 并发验收设计	48
9 实现边界与迭代计划	48
9.1 当前基线已经完成的设计落地	48
9.2 下一阶段编码优先级	48
9.3 设计变更控制	49
10 结论	49
附录 A 关键数据与接口约定	50
A.1 预约与资源状态枚举	50
A.2 HTTP 状态约定	50
A.3 代码目录与设计映射	50

1 引言与设计依据

1.1 编写目的

本设计文档用于将需求分析阶段确定的系统功能、业务规则及非功能性要求进一步转化为可实现、可联调、可测试的技术设计方案，为后续编码实现、模块集成、系统测试和版本迭代提供统一依据。

概要设计主要从系统整体层面对技术架构、模块划分、数据组织、接口关系及部署结构进行说明，明确各组成部分之间的职责边界与协作方式；详细设计则进一步描述核心业务模块的内部实现机制，包括预约流程、状态流转、时间冲突判定、并发控制、异常处理及数据持久化等关键技术细节。

本项目由五名成员并行开发，因此本文同时作为团队协同开发的技术基线，通过统一领域边界、接口契约、状态语义和数据模型，减少重复实现、跨模块强耦合以及业务规则不一致等问题，提高系统开发过程中的一致性、可维护性和可验证性。

1.2 设计依据

表 1-1 设计依据

依据	主要内容	在本文中的作用
软件需求规格说明书	用户/管理员角色、FR-01~FR-13、并发一致性、时间冲突、扩展分工	确定业务范围和验收边界
数据库表结构设计文档	8 张表、字段、外键、索引、约束	确定数据持久化模型
gitee 当前仓库	前后端目录、Domain 边界、API Contract、ReservationService、Schema	校准设计与真实实现的一致性
核心业务契约	四种持久化预约状态、半开区间、PENDING 占用、行锁顺序	冻结关键业务语义

1.3 设计原则

本系统围绕会议室资源预约场景中的模块协作、并发一致性、状态管理及后续扩展需求，遵循以下设计原则。

（1）高内聚、低耦合原则。

系统按照业务职责划分为 identity、resource、reservation、meeting 和 administration 五个业务域，各模块内部集中维护本领域的数据与业务规则。跨领域操作通过公开 Service 完成，禁止业务模块绕过领域边界直接修改其他模块拥有的数据，以降低模块之间的依赖程度，并支持小组成员并行开发与独立测试。

（2）服务端一致性优先原则。

会议室预约本质上属于有限共享资源在时间维度上的并发分配问题。前端展示的“空闲”“可预约”等信息仅作为用户交互参考，不作为预约成功的最终依据。系统以服务端事务、数据库行级锁及冲突检测结果作为资源分配的最终裁决依据，保证并发请求下预约数据的一致性。

（3）业务状态单一来源原则。

预约生命周期采用持久化状态与展示状态分离的方式。数据库字段 reservation.status 仅保存 PENDING、CONFIRMED、REJECTED 和 CANCELLED 四种具有业务事实意义的状态；UPCOMING、IN_USE 和 COMPLETED 根据持久化状态及当前时间动态推导，不重复写入数据库，从而避免同一业务事实存在多个状态来源。

（4）接口契约统一原则。

前后端交互统一采用 /api 接口前缀、标准响应结构、错误码及 HTTP 状态语义。前端通过 shared/api 适配层完成请求封装、DTO 转换与异常处理，使页面组件与后端数据结构保持适度隔离，降低接口调整对前端业务代码的影响。

（5）适度设计与渐进扩展原则。

系统采用与课程项目规模相匹配的模块化单体架构，在当前阶段不引入微服务、消息队列、分布式锁和复杂 RBAC 等额外基础设施。优先保证预约、冲突检测、资源管理与身份认证等核心链路稳定，在领域边界和核心业务契约稳定后，再逐步扩展审批、通知、会议执行、审计及统计分析等能力。

1.4 术语与约定

表 1-2 核心术语

术语	设计定义
有效占用预约	状态为 PENDING 或 CONFIRMED 的预约，两者均参与同一会议室的时间冲突检测。
半开区间	预约时间使用 [startTime, endTime)，相邻的 10:00-11:00 与 11:00-12:00 不冲突。
幂等请求	同一 requestId 的重复提交只产生一次业务结果，命中后返回已创建结果。
资源锁	事务中对 meeting_room 目标行执行 SELECT ... FOR UPDATE，使同一会议室的竞争创建在临界区内串行化。
展示状态	UPCOMING / IN_USE / COMPLETED，仅用于 UI，不写入数据库 status。

2 系统总体设计

2.1 系统设计目标

本系统总体采用前后端分离的模块化单体架构，目标是在保证核心预约业务正确性和并发一致性的前提下，实现结构清晰、职责明确、便于协作和可持续扩展的系统设计。

前端主要负责页面展示、用户交互、路由控制及短生命周期状态管理；后端负责身份认证、权限校验、业务规则执行、并发冲突裁决、事务控制以及数据持久化。系统通过统一接口契约实现前后端解耦，并通过明确的 Domain 边界组织后端业务模块，降低跨模块依赖。

根据课程设计规模要求，系统使用模块化单体作为主要实现形式。该架构能够在同一应用进程和统一数据库事务边界内完成会议室资源锁定、时间冲突校验和预约写入，降低分布式一致性带来的额外复杂度；同时通过 identity、resource、reservation、meeting 和 administration 等业务域划分，为五名成员并行开发、独立测试和后续功能扩展提供清晰边界。

系统设计最终需要满足以下目标：保证同一会议室在并发预约场景下不产生有效时间重叠；保证业务状态、接口语义和数据模型保持一致；保证各模块可以在低耦合前提下独立演进；并为后续审批、会议执行、审计和统计等功能扩展保留稳定的技术基础。

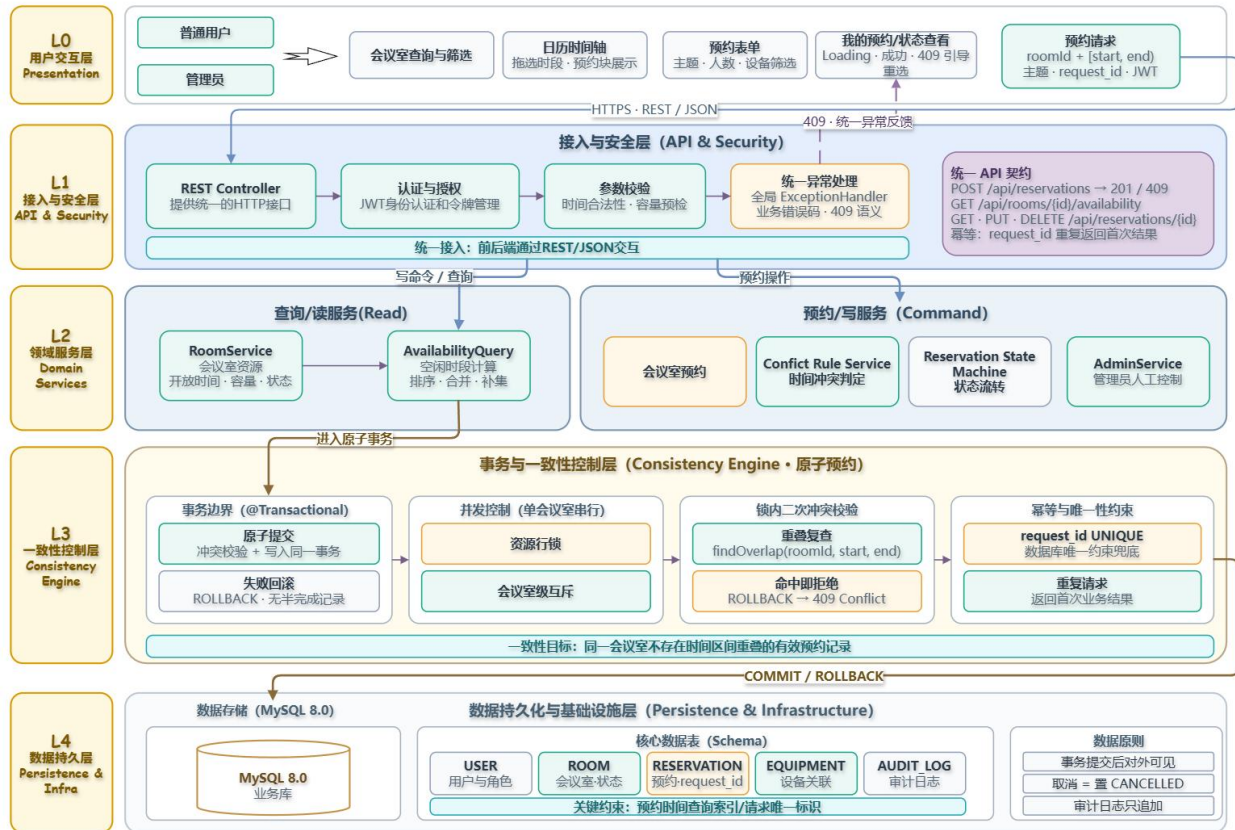


图 2-1 系统总体架构图

系统总体架构如图 2-1 所示。架构自左向右依次划分为访问与交互层、Vue 3 前端工作台、Spring Boot 应用层、核心业务域、数据持久化层以及外部能力与运维层。前端通过 REST/JSON 与后端交互，后端以五个业务域承载核心业务能力，并通过 MyBatis 访问 MySQL 数据库；其中 reservation 作为预约核心域，与资源管理、会议执行和管理审计等模块协同完成预约全生命周期处理。

2.2 技术路线

为保证系统在功能实现、并发一致性和团队协作之间取得平衡，本系统采用成熟、稳定且适合课程设计规模的技术栈。前端以 Vue 3、TypeScript 和 Vite 为基础，配合 Pinia、Vue Router 和 Element Plus 完成交互与状态管理；后端采用 Java 21、

Spring Boot、Spring Security 和 MyBatis 实现接口服务、身份认证、业务规则与数据访问；数据库采用 MySQL 8 和 InnoDB，以事务、行级锁、外键及索引支持预约场景下的数据一致性。具体技术选型及用途如表 2-1 所示。

表 2-1 技术路线

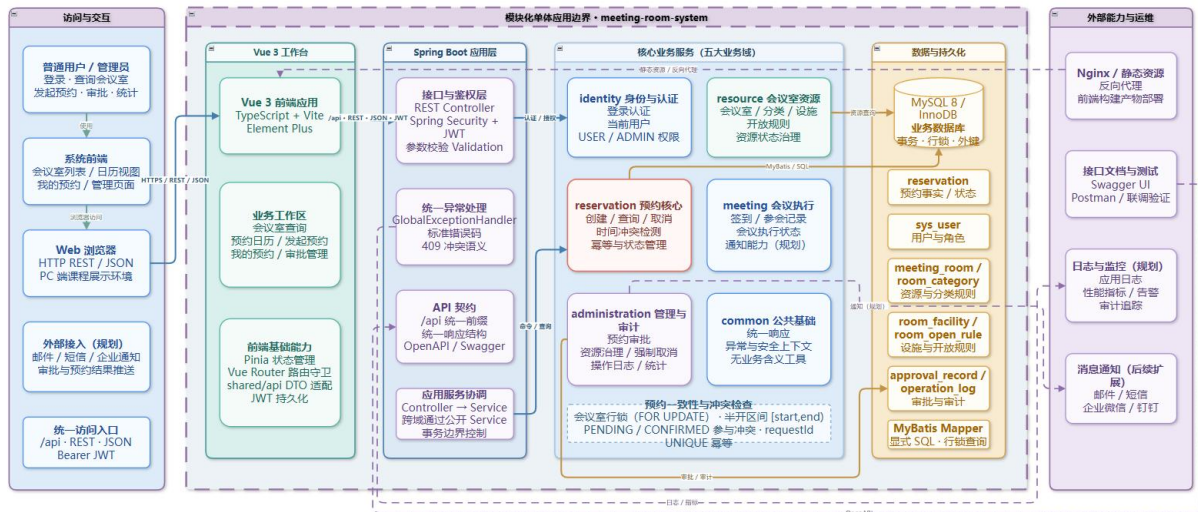
层次	技术/组件	设计用途
前端框架	Vue 3.5 + TypeScript 5.8 + Vite 6	组件化 UI、类型约束与开发构建
状态与路由	Pinia 3 + Vue Router 4.5	用户会话、会议室/预约状态、角色路由
UI 组件	Element Plus	表单、弹窗、表格、提示反馈
服务框架	Java 21 + Spring Boot 3.3.5	REST API、IOC、事务、参数校验
安全	Spring Security + JWT + BCrypt	登录认证、令牌校验、角色访问控制
持久化	MyBatis 3.0.3	显式 SQL、行锁与冲突查询可控
数据库	MySQL 8 + InnoDB	事务、行级锁、外键、CHECK 与索引
验证	Maven Test + vue-tsc + Vite build + 并发脚本	编译、单测、类型检查与并发一致性验收

上述技术选型共同构成系统的基础运行环境，其中前端负责交互与状态组织，后端负责业务裁决和事务控制，MyBatis 与 MySQL 负责数据持久化与并发一致性保障。整体方案以实现成本可控、模块职责清晰和后续扩展方便为主要考虑，不额外引入微服务、消息队列或分布式数据库等复杂基础设施。

2.3 前后端交互结构

系统采用前后端分离的交互模式，浏览器端主要承担页面展示、用户输入收集、交互状态管理和请求发起，后端负责身份认证、业务规则执行、事务控制、并发冲突处理及数据持久化。前后端通过基于 HTTP 的 REST 接口进行通信，数据统一采用

JSON 格式传输，从而保持界面实现与后端业务逻辑之间的相对独立。



浏览器端由页面组件触发用户操作。对于涉及共享状态或跨页面复用的数据，可通过 Pinia Store 或对应领域模块组织业务动作；随后 frontend/src/shared/api 中的接口适配层统一构造请求。该适配层负责屏蔽前后端 DTO 差异，例如将前端使用的 ReservationDraft 转换为后端需要的 CreateReservationRequest，并在新建预约时生成唯一的 requestId，用于后续幂等控制。

在 HTTP 通信层，系统统一完成 JWT 令牌附加、请求参数序列化、响应解析和异常转换。正常响应统一解析为 ApiResponse，非成功响应则转换为前端可识别的 ApiError，页面依据错误码完成针对性交互。例如，当后端返回 409 RESERVATION_TIME_CONFLICT 时，前端保留用户已经填写的会议信息，仅刷新当前会议室对应时段的数据，并提示用户重新选择时间。

请求进入后端后，首先经过 Spring Security 过滤链完成身份认证与授权校验，然后由 Controller 负责接收请求、完成参数绑定并调用对应领域 Service。Controller 不直接访问 Mapper，也不承担复杂业务规则。具体业务校验、事务处理、状态转换和跨领域协作统一在 Service 层完成；数据访问由 MyBatis Mapper 封装，最终通过 SQL 操作 MySQL 数据库。

系统典型调用链如下：



2.4 模块划分与依赖原则

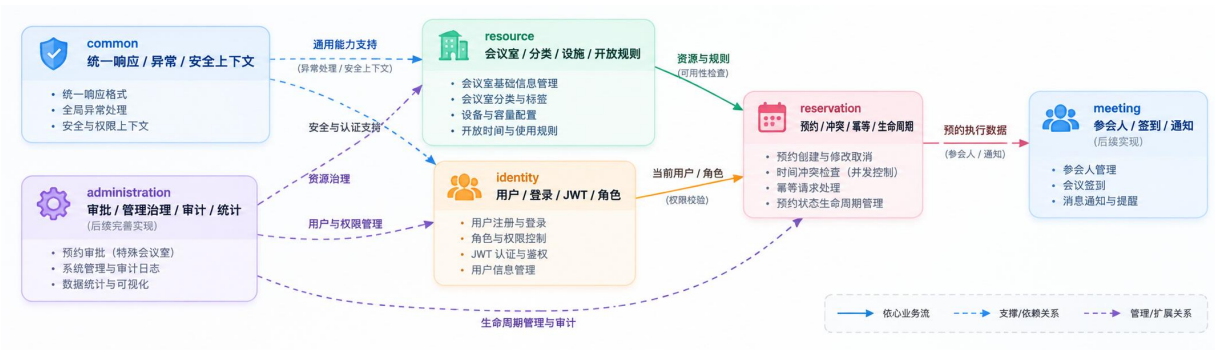


图 2-2 后端 Domain 模块依赖图

系统后端按照业务职责划分为 identity、resource、reservation、meeting 和 administration 五个主要 Domain，并设置 common 作为公共基础模块。各 Domain 负责维护自身业务规则、Service、Mapper 及对应数据表的访问逻辑，以业务能力而非技术层次作为主要模块划分依据。

其中，identity 负责身份认证、当前用户信息及角色权限；resource 负责会议室、会议室分类、设施及开放规则；reservation 负责预约创建、冲突检测、取消、幂等和预约状态生命周期；meeting 负责预约成立后的会议执行相关能力；administration 负责审批、资源治理、审计和统计等管理功能。各模块之间通过公开 Service 进行协作，不允许业务模块绕过领域边界直接访问其他 Domain 的 Mapper。

模块依赖方向表示业务服务之间的调用关系，并不代表数据层可以被任意跨域访问。系统规定每一张业务表必须具有明确的数据所有者。例如 reservation 表由预约域负责

维护，`meeting_room`、`room_category` 等资源表由资源域负责维护。若预约域需要获取会议室信息，应通过 `ResourceQueryService` 等公开服务完成，而不能直接调用 `MeetingRoomMapper`。对于跨域写操作，则必须由数据拥有者对应的 `Service` 统一封装和执行。

该依赖规则可以避免多个模块同时修改同一类数据所造成的业务语义不一致。例如，`meeting` 模块即使需要根据签到情况处理会议执行过程，也不应直接更新 `reservation.status`，而应通过预约域提供的生命周期服务完成状态相关操作；`administration` 在执行审批时，也应调用预约域提供的状态转换能力，而不是直接修改预约表。

`common` 模块仅用于存放统一响应结构、异常定义、安全上下文、通用配置以及与具体业务无关的工具类。原则上，任何包含明确业务含义的规则都不应放入 `common`。通过这一约束，可以避免公共模块逐渐演化为所有业务模块共同依赖的“公共杂物层”，从而降低全局耦合。

系统整体依赖遵循以下原则：上层模块通过公开接口调用下层能力，数据访问由数据所有者控制；跨域读操作优先通过查询 `Service` 完成，跨域写操作必须通过数据所有者 `Service` 执行；`Controller` 不直接访问 `Mapper`；业务规则不得在不同模块重复实现。对于时间冲突判断、预约状态转换和权限校验等核心规则，应保持唯一实现来源。

该模块划分方式既保证了模块内部的高内聚，也限制了跨模块耦合范围。对于五人并行开发场景，不同成员可以围绕各自业务域独立完成 `Controller`、`Service`、`Mapper` 和前端模块开发，仅通过稳定的 `Service` 与 `API` 契约进行协作，从而降低代码冲突和联调成本，并为后续功能扩展提供清晰边界。

2.5 部署与运行结构

系统开发环境采用“浏览器 + Vite 开发服务器 + Spring Boot 应用服务 + MySQL 数据库”的四段式运行结构。浏览器负责加载前端页面并发起业务请求；Vite 开发服务器默认监听 5173 端口，负责前端资源热更新及本地开发代理；Spring Boot 后端默认监听 8080 端口，对外提供 `/api` REST 接口；MySQL 默认使用 3306 端口保存用户、会议室、预约、审批及审计等业务数据。

在开发模式下，前端通过 Vite Proxy 将 `/api` 请求转发至 Spring Boot 服务，从而避免前端代码直接绑定后端实际地址，并简化本地跨域配置。后端通过环境变量或外部配置文件读取数据库连接地址、数据库账号、数据库密码及 `JWT_SECRET` 等敏感参数，避免将运行环境相关信息硬编码在源码中。

课程展示或正式部署环境中，前端通过 `npm run build` 生成静态资源文件，并部署至 Nginx 或其他静态 Web 服务器。用户访问前端页面后，浏览器直接加载构建后的静态资源；涉及业务操作的 `/api` 请求由反向代理统一转发至 Spring Boot 应用。通过该方式可以保持前后端访问入口一致，同时避免在浏览器端暴露后端服务地址和内部端口信息。



系统当前采用单实例 Spring Boot 与单实例 MySQL 的部署方式，与课程项目的业务规模和并发需求相匹配。预约一致性主要依赖数据库事务、会议室行级锁和冲突检测机制完成，因此当前阶段无需引入数据库分片、分布式锁或独立消息中间件。后续若系统访问规模扩大，可在保持业务接口与 Domain 边界不变的基础上，逐步引入容器化部署、缓存、负载均衡和监控组件。

3 系统模块详细设计

本章在系统总体架构、接口契约和数据模型已经确定的基础上，对各业务模块的内部组成、职责边界、主要组件、调用关系、数据所有权及扩展约束进行细化设计。系统采用模块化单体结构，以业务领域而非技术层次作为主要拆分依据，将后端划分为 identity、resource、reservation、meeting、administration 五个业务域，并以 common 提供统一响应、异常、安全上下文和通用配置等基础能力。模块之间通过公开 Service 协作，数据访问由数据所有者统一封装，避免跨域直接访问 Mapper 或绕过业务规则修改数据。

第三章重点回答“每个模块内部如何组成以及模块之间如何协作”。具体的半开区间冲突算法、会议室级行锁、幂等三层防护和完整事务执行顺序在第 6 章进一步展开，从而避免模块结构说明与核心业务算法说明重复。

3.1 模块总览

3.1.1 模块划分依据

模块划分遵循高内聚、低耦合和数据所有权明确三项原则。identity 围绕用户身份和认证上下文组织；resource 围绕可预约会议室及其分类、设施和开放规则组织；reservation 围绕预约事实、时间冲突与生命周期组织；meeting 承接预约成立后的会议执行阶段能力；administration 聚合管理员审批、治理、审计和统计能力。common 不拥有独立业务事实，仅提供可跨模块复用的无业务含义基础设施。

这种划分方式使业务规则尽可能靠近其数据所有者。例如，会议室状态及开放规则由 resource 统一解释，预约状态与冲突语义由 reservation 统一解释，审批模块只能通过 reservation 的生命周期服务改变预约状态。这样可以防止同一规则在多个模块中形成不同实现。

表 3-1 模块职责与实现状态

模块	核心职责	数据所有权	当前状态
identity	登录认证、JWT、当前用户解析、USER/ADMIN 授权	sys_user	核心认证链路已实现

模块	核心职责	数据所有权	当前状态
resource	会议室、分类、设施、开放规则查询与资源治理	room_category、meeting_room、room_facility、room_open_rule	查询能力已实现，管理 CRUD 后续完善
reservation	预约创建、查询、取消、冲突检测、幂等控制、状态生命周期	reservation	核心预约链路已实现
meeting	参会、签到、通知及会议执行阶段能力	后续 meeting 相关业务表	模块结构预留，业务能力后续实现
administration	审批、强制治理、操作日志、管理查询与统计	approval_record、operation_log	Domain 边界已划分，完整接口后续实现
common	统一响应、错误码、异常、安全上下文、配置及通用工具	无独立业务表	公共基础设施已实现



图 3-1 后端业务域协作关系图

如图 3-1 所示，reservation 位于核心业务协作位置。创建预约时，reservation 从 identity 获取可信用户上下文，从 resource 获取会议室及规则并在必要时请求资源锁定；审批和治理场景由 administration 调用 reservation 的生命周期能力；meeting 在执行阶段读取已确认预约并围绕会议过程扩展。箭头表示公开服务调用关系，不代表允许直接操作对方数据表。

3.1.2 数据所有权与跨域访问原则

每张业务表必须归属一个明确 Domain。数据所有权包含字段语义解释权、写入规则和生命周期控制权，而不仅是 Mapper 所在目录。例如 meeting_room 由 resource 拥

有，即使 reservation 在事务中需要锁定会议室，也应调用 resource 对外提供的 lockRoom(roomId) 能力，而不在预约域中直接编写 meeting_room 的锁 SQL。

跨域读操作优先通过查询 Service 完成；跨域写操作必须调用数据拥有者提供的写服务。Controller 仅负责 HTTP 接入和参数绑定，不直接跨域访问 Mapper。该约束既能防止业务规则分散，也使五名成员可以围绕各自领域独立开发和测试。

3.2 identity 身份与认证模块

3.2.1 模块职责与安全边界

identity 模块负责账号认证、JWT 令牌签发与解析、当前用户上下文建立以及 USER/ADMIN 两级角色授权。该模块拥有 sys_user 表，是账号状态、密码摘要和角色信息的唯一业务入口。其他模块不接收客户端传入的 userId 作为身份依据，而通过 CurrentUserProvider 获取由 Spring Security 建立的可信身份。

用户密码只保存 BCrypt 摘要。登录接口属于公开接口；除登录外的受保护接口必须先通过 Spring Security 过滤链。前端菜单和按钮显隐只用于交互体验，真正的权限判断仍由后端执行。

3.2.2 内部组件与职责

表 3-2 identity 模块内部组件

内部组件	主要职责	设计约束
AuthController	接收登录请求并返回认证结果	只负责接口接入，不直接访问 Mapper
AuthService	组织用户查询、密码校验、账号状态检查和令牌签发	认证规则集中于 Service
JwtAuthenticationFilter	解析 Bearer Token，恢复用户身份并写入安全上下文	只处理认证，不承载领域业务
JwtService	生成、验证 JWT，并解析过期时间和 Claims	JWT_SECRET 由外部配置提供
CurrentUserProvider	向业务 Service 提供 userId、username、role	禁止业务层信任前端 userId
UserMapper	读取 sys_user 及必要的用户信息	其他 Domain 不直接写 sys_user

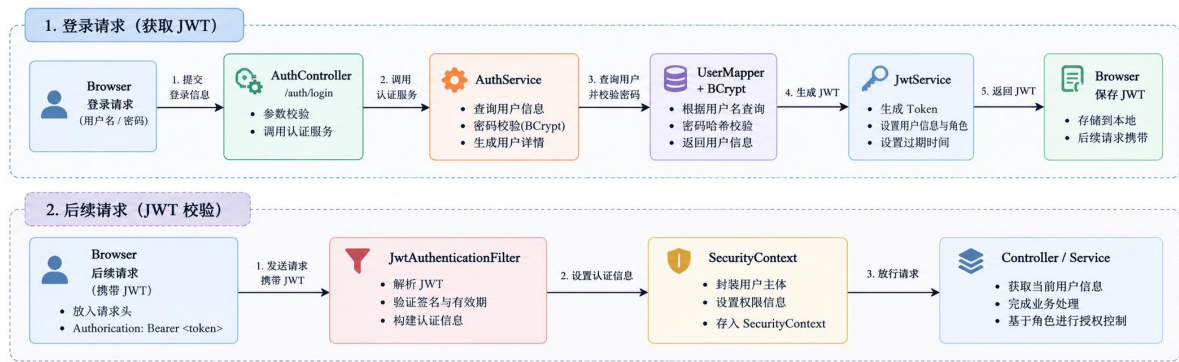


图 3-2 登录认证与后续请求处理流程图

图 3-2 登录认证与后续鉴权流程

3.2.3 登录流程设计

登录流程由 AuthController 接收 username 和 password，AuthService 根据 username 查询 sys_user 并检查账号是否启用，随后使用 BCrypt 对明文密码与数据库摘要进行匹配。认证成功后由 JwtService 生成 JWT，响应中返回 Token 与必要的当前用户信息；认证失败统一返回认证错误，不向客户端暴露密码校验细节。

登录成功后的业务请求在 Authorization 请求头中携带 Bearer JWT。JwtAuthenticationFilter 在请求进入 Controller 前完成签名验证、过期检查和身份恢复，并将认证结果写入 SecurityContext。业务 Service 通过 CurrentUserProvider 获取当前用户，因此创建预约和取消预约等操作不需要客户端重复提交 userId。

3.2.4 权限模型与异常语义

表 3-3 identity 权限控制矩阵

场景	权限要求	失败语义
登录	公开	凭据错误或账号异常：401
查询会议室、创建预约	USER / ADMIN	未登录或令牌无效：401
取消本人预约	USER / ADMIN，且需所有权 校验	非本人且非 ADMIN：403
会议室治理、审批、管理查询	ADMIN	角色不足：403

当前项目只采用 USER/ADMIN 两级角色，不引入权限表、角色继承和细粒度资源策略，以避免课程项目过度设计。若后续确需扩展部门管理员或资源管理员，可在保持 CurrentUserProvider 和 Service 授权入口不变的前提下逐步扩展。

3.3 resource 会议室资源模块

3.3.1 模块职责与对象组成

resource 模块负责描述“哪些会议室可以被预约，以及预约这些会议室需要满足哪些资源约束”。meeting_room 表保存会议室名称、位置、容量和资源状态；room_category 保存分类级审批、最大预约时长和提前预约天数；room_facility 保存会议室设施；room_open_rule 保存按星期定义的开放时间。四类对象共同构成预约前的资源约束上下文。

表 3-4 resource 领域对象与作用

资源对象	主要内容	在预约中的作用
meeting_room	名称、位置、容量、状态、分类	决定目标资源、容量上限和是否可预约
room_category	分类名称、审批要求、最大时长、提前天数	决定初始状态及分类级业务约束
room_facility	设施名称、数量	用于资源筛选与展示
room_open_rule	星期、开放/关闭时间、enabled	限定可预约时间范围

3.3.2 查询服务与管理服务分离

resource 对外能力分为查询服务和管理服务两类。ResourceQueryService 面向预约和前端查询场景，提供会议室列表、分类规则、开放规则和资源锁定等能力；ResourceManagementService 面向 ADMIN，负责新增、修改、维护、停用和恢复资源。读写能力分离有利于控制跨域依赖范围，并使普通业务域无法直接执行资源治理。

表 3-5 resource 模块服务能力

服务能力	输入	输出/效果	主要约束
资源查询	状态、分类等条件	会议室及分类、设施信息	查询结果仅用于展示或后续业务判断
lockRoom(roomId)	会议室 ID	加锁后的会议室记录	事务内 SELECT ... FOR UPDATE
getCategory(categoryId)	分类 ID	approvalRequired、maxDurationMinutes、advanceDays	分类规则统一由 resource 解释
getOpenRule(roomId, weekday)	会议室 ID、星期	开放起止时间与 enabled	用于预约时间合法性校验
资源治理	新增、修改或状态切换请求	资源变更结果	仅 ADMIN，且需处理未来预约影响

3.3.3 资源状态与治理规则

`meeting_room` 的状态映射为 `AVAILABLE`、`MAINTENANCE` 和 `DISABLED`。`AVAILABLE` 表示允许新的预约请求进入资源分配流程；`MAINTENANCE` 表示会议室暂不可用；`DISABLED` 表示资源停用。预约创建时必须在会议室行锁之后重新检查状态，防止前端曾显示可用但管理员已在并发过程中切换状态。

管理员将会议室从 `AVAILABLE` 切换到 `MAINTENANCE` 或 `DISABLED` 时，不能只修改 `meeting_room.status`。管理服务还应查询未来 `PENDING/CONFIRMED` 预约并给出处理提示，由管理员决定是否阻止状态切换、统一取消或逐条治理。当前课程基线可优先实现提示和显式处理，不采用后台自动批量修改预约的隐式策略。

3.3.4 开放规则与资源筛选

`room_open_rule` 以 `weekday` 维度维护会议室的固定开放时间。`v1.1` 设计中同一会议室同一星期只允许一个连续开放区间，因此 `(room_id, weekday)` 唯一。前端可基于这些规则提示用户可选时间，但最终合法性校验仍在后端事务流程中执行。后续若需要支持午休、分段开放等复杂场景，可将单条连续规则演进为多区间模型，而无需改变 `reservation` 对外的规则查询接口。

3.4 reservation 预约核心模块

3.4.1 模块定位与核心不变量

`reservation` 是系统核心业务域，负责把用户的预约意图转换为具有持久化和事务语义的资源占用事实。模块拥有 `reservation` 表，并统一实现预约创建、日历查询、我的预约、取消、幂等控制、状态生命周期和时间冲突规则。其最重要的不变量是：同一会议室在任意时间点，不应存在时间区间重叠的有效预约记录。

有效预约状态仅包括 `PENDING` 和 `CONFIRMED`。`REJECTED` 和 `CANCELLED` 属于历史事实，不再占用资源。`UPCOMING`、`IN_USE`、`COMPLETED` 为展示状态，由 `CONFIRMED` 和当前时间动态推导，不写入 `reservation.status`。

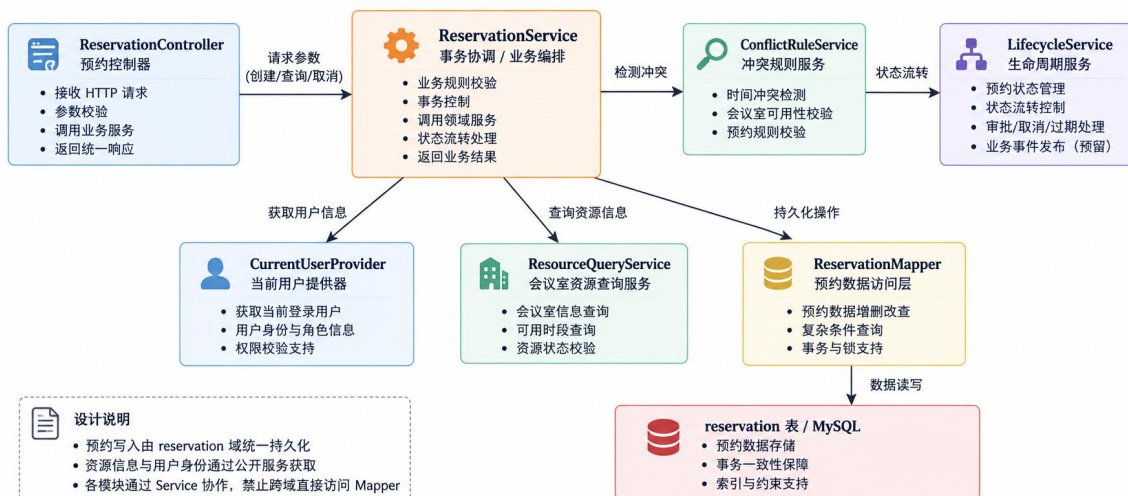


图 3-3 reservation 模块内部结构与调用关系图

图 3-3 reservation 模块内部组件关系图

3.4.2 内部组件设计

表 3-6 reservation 模块内部组件

组件	职责	关键设计点
ReservationController	提供创建、日历查询、我的预约、取消等 HTTP 入口	不直接访问 Mapper，不决定最终状态
ReservationService	组织预约事务、规则校验、冲突判断、幂等处理与取消逻辑	核心写操作统一事务边界
ReservationMapper	requestId 查询、冲突查询、插入、状态更新、列表查询	只承担数据访问，不承担业务裁决
TimeInterval	封装 startTime、endTime	保证 start < end，统一时间区间语义
ConflictRuleService	统一时间重叠规则与有效状态集合	避免空闲查询、创建、改期各写一套冲突逻辑
ReservationLifecycleService	封装审批、取消等状态转换	供 administration 和后续 meeting 调用
ReservationResponse	组合业务数据并推导 displayStatus	展示状态不进入写接口

3.4.3 创建预约命令设计

创建预约属于命令操作，由 ReservationController 接收 CreateReservationRequest 后交给 ReservationService。前端请求中包含 requestId、roomId、title、startTime、endTime、participantCount 和 remark，但不提交 userId 与最终 status。userId 来自 CurrentUserProvider，初始状态由 room_category.approval_required 决定。

ReservationService 负责协同 identity 和 resource 能力，并最终调用 ReservationMapper 写入预约。组件关系如图 3-3 所示。资源锁、冲突检测、requestId 幂等检查和插入必须处于受控事务边界；这些步骤的具体执行顺序和失败回滚策略在第 6 章详细说明。

表 3-7 创建预约请求字段职责

输入字段	来源	主要校验/用途
requestId	前端 API 适配层生成	幂等键；相同请求重复提交返回既有结果
roomId	用户选择	定位会议室并获取资源规则
title	用户输入	会议主题，不能为空且限制长度
startTime / endTime	用户选择	构造半开时间区间并执行规则与冲突校验
participantCount	用户输入	不得超过 meeting_room.capacity
remark	用户输入	可选申请备注

3.4.4 查询模型设计

预约读能力主要包括日历查询和“我的预约”。日历查询以时间范围和可选 roomId 为条件，返回前端构建时间轴所需的预约信息；“我的预约”通过当前认证用户过滤预约，并按时间排序。读模型可以组合 roomName、userName 等展示字段，但不得通过查询路径隐式修改预约状态。

空闲时段和可用会议室查询属于衍生查询能力，必须复用与创建预约相同的半开区间语义和有效状态集合。其结果用于减少用户无效尝试，但不能替代创建接口中的最终冲突裁决。

3.4.5 取消与生命周期设计

取消操作只允许作用于尚未开始的 PENDING 或 CONFIRMED 预约。普通用户必须通过所有权校验，只能取消本人预约；ADMIN 可执行治理性取消。取消采用逻辑状态变更而非物理删除，同时记录 cancel_reason，以保留预约历史和后续审计依据。

特殊会议室预约创建后进入 PENDING，并立即参与冲突检测。管理员审批通过后执行 PENDING → CONFIRMED，驳回执行 PENDING → REJECTED。PENDING 被驳

回不得用 CANCELLED 表示，以保持“审批结果”和“用户主动取消”两种业务语义可区分。

3.4.6 异常与一致性边界

表 3-8 reservation 主要业务异常

异常场景	错误语义	处理原则
会议室不存在	RESOURCE_NOT_FOUND / 404	终止事务，不创建预约
会议室维护或停用	ROOM_UNAVAILABLE / 409	锁后再次检查资源状态
参与人数超过容量	ROOM_CAPACITY_EXCEED / 400 或 409	拒绝创建并保留前端输入
时间区间重叠	RESERVATION_TIME_CONFLICT / 409	回滚事务并提示刷新日历
状态不允许操作	RESERVATION_INVALID_STATE / 409	阻止非法取消、审批或改期
requestId 语义冲突	DUPLICATE_REQUEST / 409	拒绝同一幂等键对应不同业务请求

3.5 meeting 会议执行模块

3.5.1 模块定位与实现边界

meeting 模块面向“预约成立之后”的会议执行阶段，用于承接参会人员、签到、通知、实际到场信息和会议执行记录等能力。reservation 关注资源是否被合法占用，meeting 关注会议实际执行过程，两者在时间上衔接但数据职责不同。

当前基线阶段不为尚未实现的能力提前创建空表，meeting 仅保留清晰的 Domain 位置和扩展接口。后续实现时，应以已存在的 reservation 作为会议执行上下文，不允许 meeting 直接修改 reservation.status。任何需要影响预约生命周期的行为均通过 ReservationLifecycleService 完成。

3.5.2 规划能力与协作方式

表 3-9 meeting 模块规划能力

能力	设计说明	与 reservation 的关系
参会人管理	维护会议参与成员、邀请信息等	引用预约作为会议上下文，不改变预约占用事实
签到	记录签到时间与实际到场	不直接修改预约状态
会议通知	在预约确认或临近开始时产生通知信息	读取预约时间、会议室和参与人信息

能力	设计说明	与 reservation 的关系
执行记录	记录实际开始、结束及必要执行信息	如需生命周期变更则调用 reservation 服务

3.5.3 防止执行域越权

meeting 可能天然需要读取 reservation、sys_user 和 meeting_room 等信息，因此最容易出现跨域直接查询的问题。设计上应优先通过查询服务或面向会议执行场景的聚合查询获取数据。即使为了性能后续引入只读联表查询，也应将其视为查询模型，不允许通过该查询模型写入其他 Domain 的业务表。

3.6 administration 管理与审计模块

3.6.1 模块职责与数据所有权

administration 负责管理员专属的业务治理，包括特殊会议室审批、强制取消、资源治理协调、操作日志和统计读取。该模块拥有 approval_record 与 operation_log，但并不拥有 reservation 或 meeting_room。因此审批和资源治理的实际状态修改必须调用 reservation 或 resource 的公开 Service。

3.6.2 审批服务设计

ApprovalService 接收 reservationId、action 和 remark。服务首先校验当前用户 ADMIN 权限，再读取目标预约并确认其状态为 PENDING。action=APPROVE 时调用 ReservationLifecycleService 完成 PENDING → CONFIRMED；action=REJECT 时完成 PENDING → REJECTED，并要求驳回意见非空。状态修改与 approval_record 写入应在同一事务中完成，避免出现预约状态已改变但审批历史缺失的情况。

表 3-10 administration 管理动作矩阵

管理动作	目标对象	调用服务	审计要求
审批通过/驳回	PENDING 预约	ReservationLifecycleService	写 approval_record，并记录关键管理行为
强制取消	PENDING / CONFIRMED 预约	ReservationLifecycleService	写 operation_log
会议室维护/停用	meeting_room	ResourceManagementService	写 operation_log，并检查未来预约
会议室新增/修改	meeting_room 及其规则	ResourceManagementService	写 operation_log

管理动作	目标对象	调用服务	审计要求
统计查询	预约与资源只读数据	只读统计查询服务	不修改业务数据

3.6.3 operation_log 审计设计

operation_log 只记录具有治理和责任追踪价值的关键动作，不记录所有普通 HTTP 请求。日志采用只增不改策略，通过 user_id 标识操作者，通过 operation_type 描述动作类型，通过 business_type + business_id 定位业务对象，并使用 content 保存必要的业务摘要。HTTP 访问日志和应用运行日志属于基础设施日志，不应与业务审计日志混为一体。

3.6.4 统计读取设计

统计看板属于只读能力，可基于预约量、会议室使用率、热门资源和取消率等指标构建查询模型。统计接口不得成为绕过领域服务修改业务数据的入口。对于课程规模，优先采用数据库聚合查询和必要索引，不提前引入独立数据仓库或消息流处理。

3.7 前端模块设计

3.7.1 前端分层结构

前端采用 Vue 3 + TypeScript，并按视图、领域模块、共享状态、API 适配和路由职责组织。页面组件只处理展示、用户输入和局部交互；modules 按 identity、resource、reservation、meeting、administration 对应业务域组织复用逻辑和类型；Pinia Store 保存需要跨组件共享的短生命周期状态；shared/api 集中完成 HTTP、JWT、DTO 映射和错误转换。

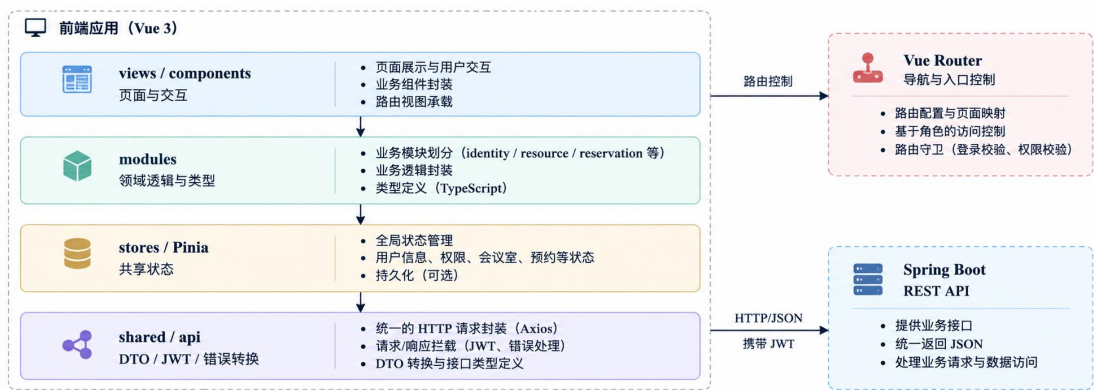


图 3-4 前端整体架构与模块关系图

图 3-4 前端分层与 API 适配结构

表 3-11 前端分层职责

前端层	设计责任	主要约束
views / components	会议室查询、预约日历、预约表单、我的预约、管理页面等 UI	不直接拼接后端 URL，不承担最终业务裁决
modules	按业务域组织组件、组合逻辑和 TypeScript 类型	概念与后端 Domain 保持一致
stores / Pinia	当前用户、会议室、预约等共享页面状态	前端状态不是服务端业务事实的权威来源
shared/api	统一 request、JWT、DTO 适配、ApiResponse 解析、ApiError 转换	真实 HTTP 调用集中封装
router	页面导航、登录态检查和管理员入口控制	前端路由守卫不替代后端权限校验

3.7.2 页面与业务模块映射

表 3-12 页面与业务模块映射

页面/视图	主要业务模块	主要数据来源
登录页	identity	/api/auth/login
会议室列表/筛选	resource	/api/rooms
预约日历	reservation + resource	/api/reservations/calendar、会议室信息
新建预约表单	reservation + resource	会议室规则、创建预约接口
我的预约	reservation	/api/reservations/my
管理页面	administration + resource + reservation	审批、资源治理、管理查询接口

3.7.3 API 适配与 DTO 转换

前端不直接以页面状态对象作为后端请求 DTO。以创建预约为例，页面形成 ReservationDraft，shared/api 在发送前转换为 CreateReservationRequest，同时生成 requestId、附加 Authorization 请求头并统一处理时间格式。响应成功后将 ApiResponse.data 转换为前端领域类型；非成功响应则转换为 ApiError，由页面依据 code 做精确反馈。

DTO 适配层能够隔离后端字段变化。例如后端新增 reservationNo 或 displayStatus 时，只需要调整 shared/api 与前端领域类型映射，而不是修改所有页面组件。Mock 与真实 HTTP 也可以通过同一适配接口切换，避免组件直接感知数据来源。

3.7.4 冲突反馈与局部刷新

当创建预约返回 409 RESERVATION_TIME_CONFLICT 时，前端保留 title、participantCount、remark 等已经输入的内容，仅清晰提示时段冲突，并重新请求当前日期和会议室的日历数据。用户可以在原表单中重新选择时间或会议室。该策略避免整页刷新造成上下文丢失，同时保证冲突发生后页面重新对齐服务端最新事实。

表 3-13 前端错误反馈策略

后端错误	前端处理	是否清空表单
VALIDATION_ERROR	定位参数或规则问题并显示 message	否
UNAUTHORIZED	清理失效会话并引导重新登录	视场景
FORBIDDEN	提示无权限并返回可访问页面	否
RESOURCE_NOT_FOUND	提示资源已不存在并刷新列表	否
RESERVATION_TIME_CONFLICT	提示冲突并局部刷新日历	否
INTERNAL_ERROR	显示通用失败提示并保留输入	否

3.8 模块协作与集成约束

3.8.1 跨域调用矩阵

为了控制模块耦合，系统对主要跨域调用关系进行显式约束。下表描述的是允许的服务级依赖，而不是数据库访问许可。任何新增跨域依赖均应优先判断是否能够通过现有查询服务或生命周期服务完成。

表 3-14 跨域调用矩阵

调用方	被调用方	允许的典型能力	禁止行为
reservation	identity	读取当前用户上下文	直接修改 sys_user
reservation	resource	查询资源、规则、锁定会议室	直接调用 resource Mapper
administration	reservation	审批、强制取消等生命周期操作	直接 UPDATE reservation.status
administration	resource	新增、修改、维护、停用会议室	直接操作 meeting_room 表
meeting	reservation	读取预约上下文、调用生命周期能力	绕过 reservation 修改主表
前端页面	shared/api	调用稳定 API 适配接口	在组件中散落裸 URL 与 Axios 细节

3.8.2 事务归属原则

事务边界应由发起核心业务状态变更的 Domain Service 持有。创建预约由 ReservationService 持有事务；审批由 administration 发起，但预约状态转换与 approval_record 写入需要形成单一一致性单元；资源治理由 ResourceManagementService 控制。Controller 不开启业务事务，Mapper 也不负责决定事务范围。

跨域调用中若涉及同一业务事务，应通过服务调用在同一进程内参与事务，而不是通过异步消息拆分。课程基线采用模块化单体正是为了让会议室锁定、冲突校验和预约写入能够处于统一数据库事务边界。

3.8.3 模块测试与联调边界

模块开发应先验证本域内部规则，再进行跨域联调。identity 重点测试登录、令牌与权限；resource 重点测试资源查询、规则与状态治理；reservation 重点测试创建、取消、冲突和幂等；administration 重点测试审批和审计；前端重点验证 API 适配、错误反馈和状态刷新。跨域测试通过公开 Service 或 REST API 进行，不以直接修改数据库作为正常测试路径。

表 3-15 模块测试与联调重点

模块	单模块重点	集成验证重点
identity	账号认证、JWT、角色输出	受保护接口能正确获取用户上下文
resource	规则读取、状态治理、行锁读取	reservation 能通过公开 Service 获取资源上下文

模块	单模块重点	集成验证重点
reservation	冲突、幂等、状态转换、取消	并发创建与 resource/identity 协作正确
administration	审批、强制治理、日志记录	状态变更与审计记录保持一致
frontend	DTO 适配、Pinia 状态、错误处理	真实 API 下页面与服务端状态保持同步

3.8.4 面向五人并行开发的协作约束

五名成员可分别围绕主要业务域承担前后端完整开发，但公共业务契约必须统一维护。对 `reservation.status`、时间区间语义、`approval_required`、错误码和数据库所有权等核心契约的修改，不应由单个成员在本地模块中直接改变。此类变更需要同步更新后端枚举与 `Service`、前端类型与 `API` 适配、数据库 `migration` 和相关测试。

模块接口尽量保持稳定，新增需求优先通过增加 `Service` 方法或 `DTO` 字段演进，而不是绕开现有边界形成临时跨域访问。这样可以降低多人同时开发时的代码冲突和联调成本，并保证后续扩展审批、会议执行和统计能力时仍能维持当前架构。

3.9 本章小结

本章完成了系统模块级详细设计。`identity` 建立可信用户上下文和角色授权；`resource` 统一维护会议室及其分类、设施与开放规则；`reservation` 作为核心域负责预约事实、冲突语义、幂等和生命周期；`meeting` 承接会议执行阶段扩展；`administration` 负责审批、治理和审计；前端通过领域模块、`Pinia Store` 和 `shared/api` 与后端保持对应边界。

通过明确数据所有权、公开 `Service` 调用、事务归属和前端 `API` 适配规则，系统在保持模块高内聚的同时限制了跨域耦合。上述模块结构为后续数据库设计、接口实现以及第 6 章的预约流程、并发控制和状态机详细设计提供了统一基础。

4 数据库设计

4.1 数据库总体设计

数据库采用 `MySQL 8.x InnoDB`，字符集 `utf8mb4`，核心实体使用 `BIGINT` 自增主键，时间使用 `DATETIME`。业务状态不使用 `MySQL ENUM`，而使用 `VARCHAR/TINYINT` 配合应用层枚举维护，以降低数据库与业务状态机的耦合。数据

库主要保证引用完整性、唯一性和基本字段合法性；跨表业务规则和时间区间冲突由 Service 在事务中完成。

表 4-1 数据表清单

序号	表名	中文名称	主要职责
1	sys_user	系统用户表	登录账号、角色、联系方式
2	room_category	会议室分类表	类型、审批/时长/提前预约规则
3	meeting_room	会议室表	具体会议室资源与容量
4	room_facility	会议室设施表	设施及数量
5	room_open_rule	会议室开放规则表	每周可预约时间
6	reservation	预约表	核心预约事实、时间区间、状态
7	approval_record	审批记录表	管理员审批/驳回行为历史
8	operation_log	操作日志表	关键管理操作与审计

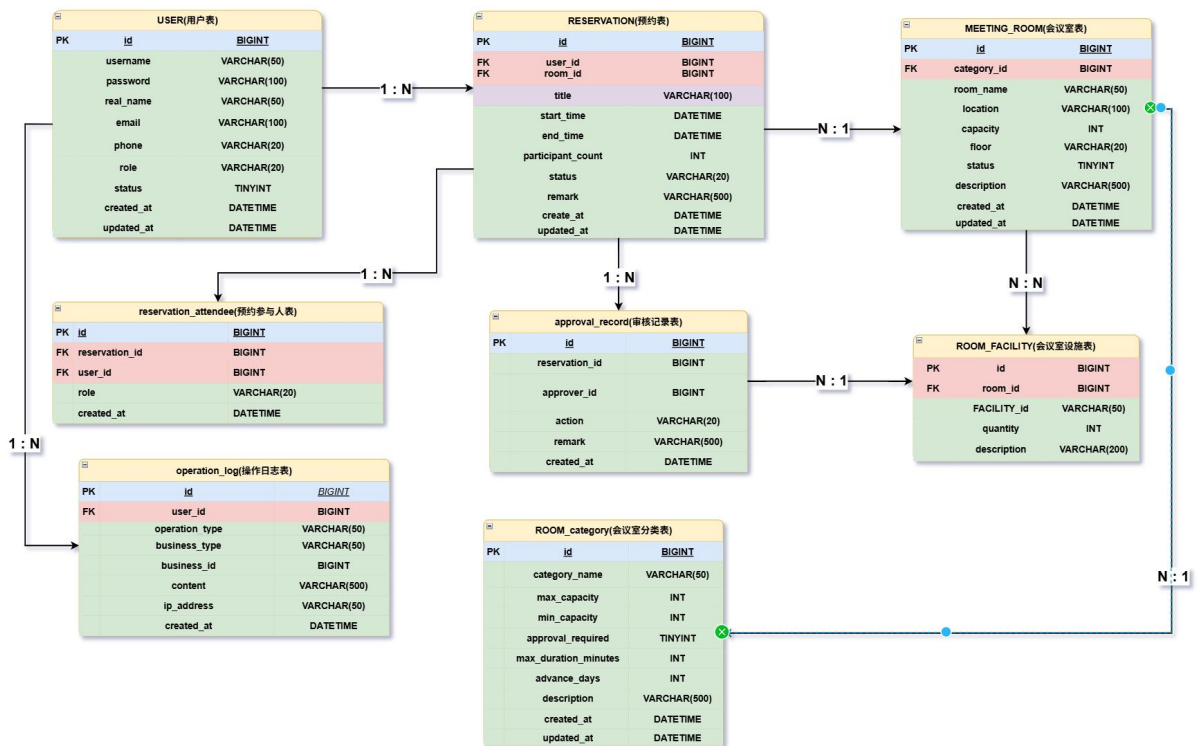


图 4-1 数据库主要实体关系图

4.2 主键、外键与关系设计

room_category 与 meeting_room 为一对多；meeting_room 与 room_facility、room_open_rule、reservation 均为一对多；sys_user 与 reservation 为一对多；reservation 与 approval_record 为一对多；sys_user 还分别以 approver_id 和 user_id 与 approval_record、operation_log 建立关系。operation_log.business_id 允许引用不同类型的业务对象，因此采用 business_type + business_id 的多态业务引用，不设置跨业务表外键。

4.3 数据表字段设计

4.3.1 sys_user

保存普通用户与管理员账号。用户名唯一，password 仅保存 BCrypt 摘要；当前课程规模采用 USER/ADMIN 两级角色，不引入复杂 RBAC。

表 4-2 sys_user 字段结构

字段名	类型	约束	默认值	说明
id	BIGINT	PK, AUTO_INCREMENT	-	用户主键 ID
username	VARCHAR(50)	UNIQUE, NOT NULL	-	登录用户名
password	VARCHAR(100)	NOT NULL	-	BCrypt 密码摘要
real_name	VARCHAR(50)	NOT NULL	-	真实姓名
email	VARCHAR(100)	NULL	NULL	邮箱
phone	VARCHAR(20)	NULL	NULL	手机号
role	VARCHAR(20)	NOT NULL	USER	USER / ADMIN
status	TINYINT	NOT NULL	1	1 正常, 0 禁用
created_at	DATETIME	NOT NULL	CURRENT_TIMESTAMP	创建时间
updated_at	DATETIME	NOT NULL	自动更新	更新时间

4.3.2 room_category

保存会议室类型与相对稳定的预约策略，approval_required 决定初始预约状态，时长与提前天数在服务层校验。

表 4-3 room_category 字段结构

字段名	类型	约束	默认值	说明
id	BIGINT	PK, AUTO_INCREMENT	-	分类 ID
category_name	VARCHAR(50)	UNIQUE, NOT NULL	-	分类名称
min_capacity	INT	NOT NULL	-	容量下限
max_capacity	INT	NOT NULL	-	容量上限
approval_required	TINYINT	NOT NULL	0	是否需要审批
max_duration_minute s	INT	NOT NULL	240	单次预约最大时长
advance_days	INT	NOT NULL	7	允许提前预约最大 天数
description	VARCHAR(500)	NULL	NULL	分类说明

字段名	类型	约束	默认值	说明
created_at	DATETIME	NOT NULL	CURRENT_TIMESTAMP	创建时间
updated_at	DATETIME	NOT NULL	自动更新	更新时间

4.3.3 meeting_room

保存具体可预约资源。category_id 关联分类；capacity 是参与人数校验的最终资源属性；status 映射到 AVAILABLE/MAINTENANCE/DISABLED。

表 4-4 meeting_room 字段结构

字段名	类型	约束	默认值	说明
id	BIGINT	PK, AUTO_INCREMENT	-	会议室 ID
category_id	BIGINT	FK → room_category.id	-	所属分类
room_name	VARCHAR(50)	UNIQUE, NOT NULL	-	会议室名称
location	VARCHAR(100)	NULL	NULL	楼栋/楼层位置
capacity	INT	NOT NULL	-	实际容纳人数
status	TINYINT	NOT NULL	1	1 可用, 0 维护中, 2 停用
description	VARCHAR(500)	NULL	NULL	会议室说明
created_at	DATETIME	NOT NULL	CURRENT_TIMESTAMP	创建时间
updated_at	DATETIME	NOT NULL	自动更新	更新时间

4.3.4 room_facility

一条记录表示某会议室的一种设施与数量，(room_id, facility_name) 唯一，避免重复登记。

表 4-5 room_facility 字段结构

字段名	类型	约束	默认值	说明
-----	----	----	-----	----

字段名	类型	约束	默认值	说明
id	BIGINT	PK, AUTO_INCREMENT	-	设施记录 ID
room_id	BIGINT	FK → meeting_room.id	-	会议室 ID
facility_name	VARCHAR(50)	(room_id,facility_name) UNIQUE	-	设施名称
quantity	INT	NOT NULL	1	设施数量
description	VARCHAR(200)	NULL	NULL	设施说明

4.3.5 room_open_rule

按星期保存固定开放区间，v1.1 每个会议室每天一个连续区间，weekday 使用 1~7。

表 4-6 room_open_rule 字段结构

字段名	类型	约束	默认值	说明
id	BIGINT	PK, AUTO_INCREMENT	-	开放规则 ID
room_id	BIGINT	FK → meeting_room.id	-	会议室 ID
weekday	TINYINT	CHECK 1..7; 联合唯一	-	ISO 周序号
open_time	TIME	NOT NULL	-	开放起始时间（含）
close_time	TIME	CHECK close > open	-	关闭时间（不含）
enabled	TINYINT	NOT NULL	1	规则是否启用
created_at	DATETIME	NOT NULL	CURRENT_TIMESTAMP	创建时间
updated_at	DATETIME	NOT NULL	自动更新	更新时间

4.3.6 reservation

系统核心业务表。request_id 负责幂等，reservation_no 负责业务追踪；PENDING 与 CONFIRMED 参与冲突。

表 4-7 reservation 字段结构

字段名	类型	约束	默认值	说明
id	BIGINT	PK, AUTO_INCREMENT	-	预约主键 ID
request_id	VARCHAR(100)	UNIQUE, NOT NULL	-	幂等键
reservation_no	VARCHAR(32)	UNIQUE, NOT NULL	-	预约业务单号
room_id	BIGINT	FK → meeting_room.id	-	会议室 ID
user_id	BIGINT	FK → sys_user.id	-	预约人 ID
title	VARCHAR(100)	NOT NULL	-	会议主题
start_time	DATETIME	NOT NULL	-	开始时间
end_time	DATETIME	CHECK end > start	-	结束时间
participant_count	INT	NOT NULL	1	参与人数
status	VARCHAR(20)	NOT NULL	-	PENDING/CONFIRMED/REJECTED/CANCELLED
remark	VARCHAR(500)	NULL	NULL	申请备注
cancel_reason	VARCHAR(500)	NULL	NULL	取消原因
created_at	DATETIME	NOT NULL	CURRENT_TIMESTAMP	创建时间
updated_at	DATETIME	NOT NULL	自动更新	更新时间

4.3.7 approval_record

记录审批行为事实，不在 reservation 中冗余 approver_id/approved_at；允许一条预约保留多次审批历史。

表 4-8 approval_record 字段结构

字段名	类型	约束	默认值	说明
id	BIGINT	PK, AUTO_INCREMENT	-	审批记录 ID
reservation_id	BIGINT	FK → reservation.id	-	预约 ID
approver_id	BIGINT	FK → sys_user.id	-	审批管理员 ID

字段名	类型	约束	默认值	说明
action	VARCHAR(20)	NOT NULL	-	APPROVE / REJECT
remark	VARCHAR(500)	NULL	NULL	审批意见
created_at	DATETIME	NOT NULL	CURRENT_TIMESTAMP	审批时间

4.3.8 operation_log

记录关键管理行为，不记录所有 HTTP 请求；日志只增不改，business_type + business_id 定位业务对象。

表 4-9 operation_log 字段结构

字段名	类型	约束	默认值	说明
id	BIGINT	PK, AUTO_INCREMENT	-	日志 ID
user_id	BIGINT	FK → sys_user.id	-	操作人 ID
operation_type	VARCHAR(50)	NOT NULL	-	关键操作类型
business_type	VARCHAR(50)	NOT NULL	-	RESERVATION/MEETING_ROOM/ROOM_CATEGORY
business_id	BIGINT	NOT NULL	-	多态业务对象 ID
content	VARCHAR(500)	NOT NULL	-	操作内容
ip_address	VARCHAR(50)	NULL	NULL	操作 IP
created_at	DATETIME	NOT NULL	CURRENT_TIMESTAMP	操作时间

4.4 索引与约束设计

表 4-10 主要索引与约束

对象	设计	目的
sys_user	UNIQUE(username)	防止重复登录账号
meeting_room	UNIQUE(room_name), INDEX(category_id)	保证会议室命名唯一并优化分类查询
room_facility	UNIQUE(room_id, facility_name)	防止同一会议室重复设施项

对象	设计	目的
room_open_rule	UNIQUE(room_id, weekday), CHECK weekday 1..7, CHECK close_time > open_time	保证开放规则合法且每天单一
reservation	UNIQUE(request_id), UNIQUE(reservation_no)	保障幂等与业务单号唯一
reservation	INDEX(room_id, status, start_time)	缩小同房间有效预约冲突候选集
reservation	INDEX(user_id, start_time)	优化“我的预约”与用户时间排序
approval_record	INDEX(reservation_id), INDEX(approver_id)	优化审批历史与管理员查询
operation_log	INDEX(user_id), INDEX(business_type,business_i d)	优化审计查询

4.5 数据库并发一致性设计

时间区间互斥不能仅通过普通 UNIQUE 约束表示。因此数据库层只提供事务、行级锁、唯一键和高效候选查询，业务层负责执行唯一冲突条件 $newStart < oldEnd \text{ AND } newEnd > oldStart$ 。对同一会议室，创建预约先锁 meeting_room 行，再查询 reservation 冲突；对不同会议室，请求锁定不同资源行，可并行处理，避免全局 synchronized 带来的无必要串行。

5 接口设计

5.1 接口基本约定

表 5-1 接口基本约定

项目	约定
统一前缀	/api
传输格式	application/json
时间格式	ISO-8601 yyyy-MM-dd'T'HH:mm:ss
认证方式	Bearer JWT；登录接口公开，其余接口按 USER/ADMIN 授权

项目	约定
统一成功结构	<code>{"code":"SUCCESS","message":"success","data":...}</code>
统一失败结构	<code>{"code":"<ERROR_CODE>","message":"...","data":null}</code>
前端异常策略	业务错误保留表单输入、展示 message，冲突后局部刷新日历，不整页刷新

5.2 当前已实现 API

表 5-2 已实现接口清单

Method	Path	权限	用途	HTTP 成功
POST	/api/auth/login	公开	用户名/密码登录，返回 JWT 与用户	200
GET	/api/users/me	USER/ADMIN	读取当前用户	200
GET	/api/rooms	USER/ADMIN	查询会议室列表，可按状态筛选	200
GET	/api/reservations/calendar	USER/ADMIN	按 start/end/roomId 查询日历预约	200
POST	/api/reservations	USER/ADMIN	创建预约；首次创建返回 201	201/200
GET	/api/reservations/my	USER/ADMIN	查询当前用户预约	200
POST	/api/reservations/{id}/cancel	本人/ADMIN	开始前取消有效预约	200

5.3 规划 API

表 5-3 规划接口清单

Method	Path	权限	设计用途
POST	/api/rooms	ADMIN	新增会议室
PUT	/api/rooms/{id}	ADMIN	修改会议室基础信息
PATCH	/api/rooms/{id}/status	ADMIN	维护/停用/恢复资源状态
GET	/api/rooms/{id}/free-slots	USER/ADMIN	计算指定日期空闲时段
GET	/api/rooms/available	USER/ADMIN	按时间/容量/设备查询可用会议室

Method	Path	权限	设计用途
PUT	/api/reservations/{id}	本人/ADMIN	预约改期/信息修改，重新执行冲突判定
GET	/api/admin/reservations	ADMIN	管理端全量预约查询
POST	/api/admin/reservations/{id}/approval	ADMIN	APPROVE/REJECT 特殊会议室预约

5.4 创建预约接口

POST /api/reservations 是核心写接口。前端不提交 userId 和最终 status，避免越权和状态伪造；userId 从 JWT 认证上下文取得，初始 status 根据 room_category.approval_required 在服务端决定。

```
请求示例：
{
  "requestId": "req-20260914-0001",
  "roomId": 1,
  "title": "课程设计讨论",
  "startTime": "2026-09-15T10:00:00",
  "endTime": "2026-09-15T11:00:00",
  "participantCount": 6,
  "remark": "需要投影仪"
}
```

响应核心字段：id, reservationNo, roomId, roomName, userId, userName, title, startTime, endTime, participantCount, status, displayStatus, remark

5.5 错误码与 HTTP 状态语义

表 5-4 统一错误码

错误码	HTTP	触发场景
VALIDATION_ERROR	400	时间、开放规则、最大时长等参数或业务校验失败
UNAUTHORIZED	401	未登录、JWT 无效、登录凭据错误
FORBIDDEN	403	角色或资源所有权不足
RESOURCE_NOT_FOUND	404	用户、会议室或预约不存在
ROOM_UNAVAILABLE	409	会议室维护/停用或资源状态竞争
ROOM_CAPACITY_EXCEEDED	400/409	参与人数超过容量
RESERVATION_TIME_CONFLICT	409	同一房间存在重叠有效预约
RESERVATION_INVALID_STATE	409	当前状态不允许取消/审批/修改

错误码	HTTP	触发场景
DUPLICATE_REQUEST	409	requestId 与已存在请求语义不一致
INTERNAL_ERROR	500	未预期服务器异常

6 核心业务详细设计

本章围绕会议室预约系统最关键的业务闭环，对创建预约、时间冲突判定、并发控制、幂等处理、预约状态机、取消操作、特殊会议室审批以及前端冲突反馈进行详细设计。相关设计以第 3 章的 Domain 边界、第 4 章的数据模型和第 5 章的接口契约为基础，重点说明核心业务在事务边界内如何执行，以及系统如何在并发条件下保持预约数据一致。

核心业务遵循三个不变量：第一，同一会议室不存在时间区间重叠的有效预约；第二，同一 requestId 只产生一次业务结果；第三，预约持久化状态只由 reservation 领域的生命周期能力修改。有效占用状态仅包括 PENDING 和 CONFIRMED，REJECTED 与 CANCELLED 不再占用会议室资源。

6.1 创建预约程序流程

6.1.1 业务入口与输入约束

创建预约使用 POST /api/reservations 作为核心写接口。前端提交 requestId、roomId、title、startTime、endTime、participantCount 和 remark，但不提交 userId 与最终 status。userId 由当前认证上下文提供，初始状态由会议室分类的 approval_required 决定，从而避免客户端伪造预约人或业务状态。

表 6-1 创建预约核心输入及职责

字段	来源	主要用途/约束
requestId	前端 API 适配层生成	幂等键；重复提交时定位既有业务结果
roomId	用户选择	定位目标会议室并进入资源级并发控制
title	用户输入	会议主题，不能为空并受字段长度约束

字段	来源	主要用途/约束
startTime / endTime	用户选择	构造半开区间 [startTime, endTime), 要求 startTime < endTime
participantCount	用户输入	用于与 meeting_room.capacity 比较
remark	用户输入	可选申请备注, 不参与资源占用判定

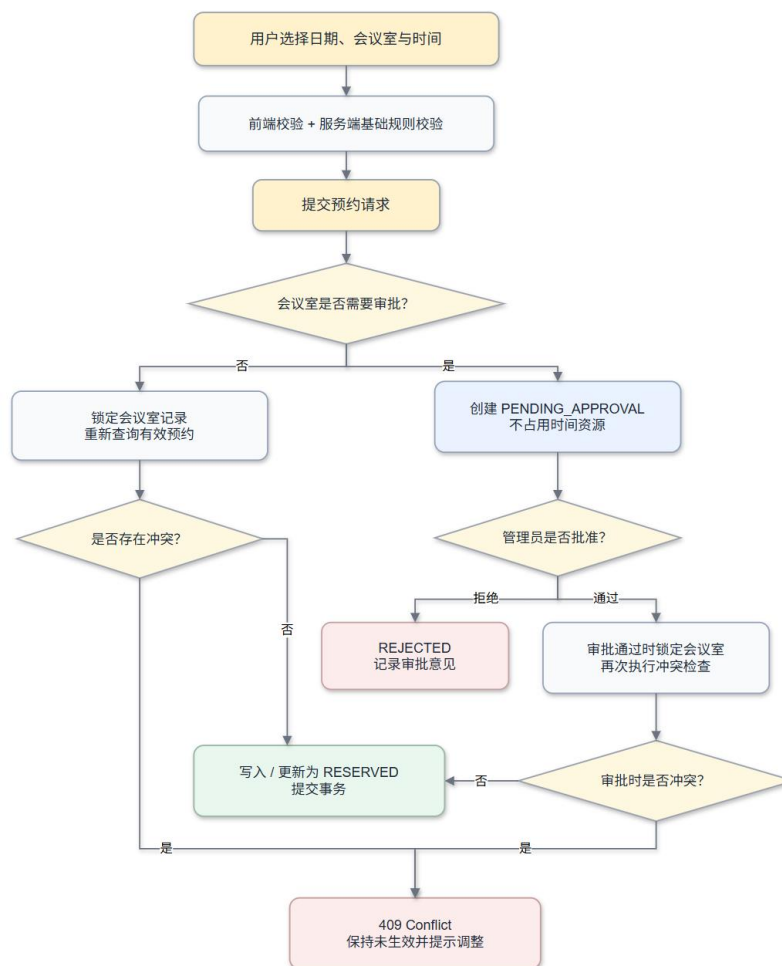


图 6-1 创建预约核心程序流程图

表 6-2 创建预约事务步骤

顺序	事务内操作	失败或命中结果
1	读取当前认证用户；首次查询 requestId	命中既有 requestId 时直接返回已创建结果
2	构造 TimeInterval，校验 start < end 且开始时间不早于当前时间	参数或时间不合法时返回 VALIDATION_ERROR

顺序	事务内操作	失败或命中结果
3	锁定 meeting_room 目标行	资源不存在时返回 404；同一 roomId 的竞争请求在此等待
4	获得会议室锁后二次查询 requestId	覆盖重复请求在锁等待期间已经完成的情况
5	检查 AVAILABLE、分类规则、容量和开放时间	规则不满足时回滚事务并返回对应业务错误
6	查询 PENDING / CONFIRMED 时间冲突	存在重叠时返回 RESERVATION_TIME_CONFLICT / 409
7	根据 approval_required 决定初始状态	普通会议室进入 CONFIRMED；特殊会议室进入 PENDING
8	插入 reservation，依赖 UNIQUE(request_id) 最终兜底	若发生唯一键竞争则回查并返回既有结果
9	提交事务	首次创建成功返回 201；幂等命中返回既有结果

6.1.3 成功结果与失败回滚

创建预约成功后，数据库中生成一条具有明确 room_id、user_id、时间区间和持久化状态的 reservation 记录。事务提交之前，其他竞争请求不会把未提交数据视为最终业务事实；事务发生任何未处理业务错误时，当前写操作回滚，不留下部分完成的预约。该设计使资源校验、冲突判定和预约写入构成单一原子业务操作。

该流程中的关键点是“锁会议室行在冲突查询之前”。事务隔离级别为 READ_COMMITTED；锁等待结束后再次查询 requestId，用于处理两个相同幂等请求几乎同时到达的情况。这样可避免等待锁的重复请求在第一个请求提交后被错误识别为时间冲突。

6.2 时间冲突判定算法

6.2.1 半开区间模型

系统统一采用半开时间区间 [startTime, endTime) 表示预约。半开区间的右端点不包含在预约占用范围内，因此 10:00-11:00 与 11:00-12:00 可以连续排期而不产生冲突。该时间语义必须同时用于创建预约、空闲时段计算、可用会议室查询和改期等所有功能，避免不同接口对边界时间作出不同判断。

新预约 $N = [N_s, N_e)$ ，已有预约 $O = [O_s, O_e)$

冲突条件： $N_s < O_e$ 且 $N_e > O_s$

上述条件等价于“两个区间既不是新预约完全位于旧预约之后，也不是新预约完全位于旧预约之前”。只要两个不等式同时成立，即说明两个时间区间存在正长度交集。冲突候选只包含 PENDING 和 CONFIRMED；REJECTED 与 CANCELLED 仅保留历史事实，不参与资源占用。

表 6-3 典型时间区间关系

场景	示例	是否冲突	判定依据
完全分离	09:00-10:00 与 10:00-11:00	否	边界相接，半开区间不重叠
新预约覆盖已有预约	09:00-12:00 与 10:00-11:00	是	$N_s < O_e$ 且 $N_e > O_s$
新预约位于已有预约内部	10:15-10:45 与 10:00-11:00	是	存在正长度交集
左侧部分重叠	09:30-10:30 与 10:00-11:00	是	新预约结束时间晚于已有开始时间
右侧部分重叠	10:30-11:30 与 10:00-11:00	是	新预约开始时间早于已有结束时间
历史预约	与 REJECTED / CANCELLED 时间重叠	否	历史状态不占用资源

6.2.2 数据库查询条件

冲突判断在服务层统一调用 ReservationMapper 的冲突查询。其逻辑条件如下，数据库索引 INDEX(room_id, status, start_time) 用于缩小同一会议室、有效状态和目标时间附近的候选集合。

```
WHERE room_id = :roomId
AND status IN ('PENDING', 'CONFIRMED')
AND start_time < :newEnd
AND end_time > :newStart
```

该查询返回的不是“是否曾经存在预约”，而是“在当前有效状态集合中是否存在时间交集”。因此冲突规则的语义来源应保持唯一，空闲时段查询和可用会议室查询也必须复用同一条件。

6.3 并发控制与事务边界

6.3 并发控制与事务边界

6.3.1 会议室级并发控制

系统选择会议室级并发控制粒度，而不是使用系统级 `synchronized` 或全局锁。创建预约时只锁定目标 `meeting_room` 行，因此竞争同一 `roomId` 的请求在资源级临界区内串行执行，而不同会议室的预约请求可以并行处理。该粒度既能保证同一资源的一致性，又避免不相关会议室之间发生无必要阻塞。

事务隔离级别使用 `READ_COMMITTED`。关键点不是依赖隔离级别本身阻止幻读，而是先通过 `SELECT ... FOR UPDATE` 对 `meeting_room` 目标行建立互斥，再在该临界区内查询 `reservation` 冲突。只要所有创建预约路径遵循相同锁定规则，同一会议室的竞争请求就会按锁获取顺序完成最终冲突裁决。

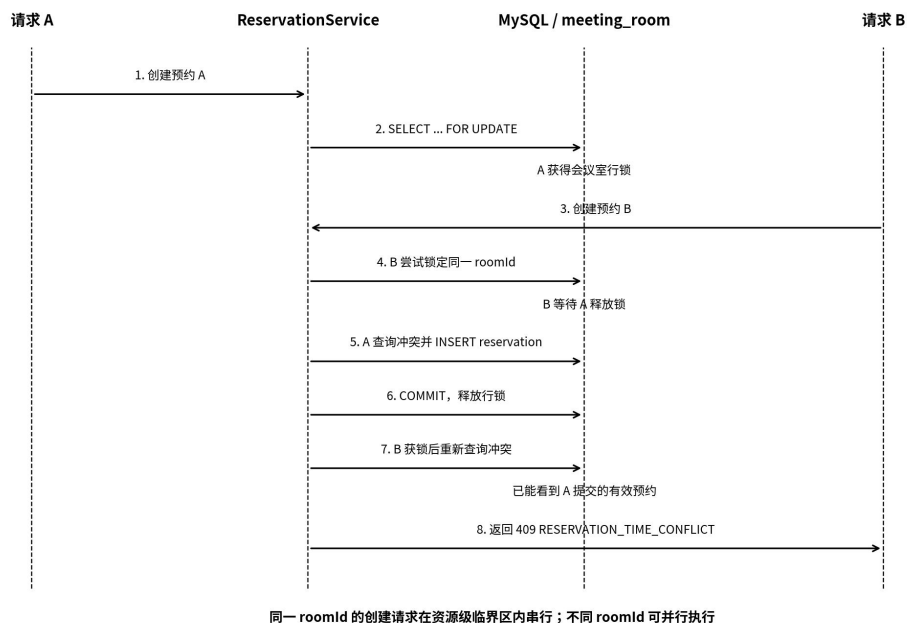


图 6-2 同一会议室并发预约控制时序图

6.3.2 并发场景分析

表 6-4 并发场景与系统行为

并发场景	系统行为	一致性结果
两个请求预约同一 roomId、相同时间段	先获得 meeting_room 行锁的事务进入冲突查询；另一请求等待	先提交者成功，后进入者重新查询后返回 409
两个请求预约同一 roomId、不重叠时间段	请求仍按会议室行锁串行进入临界区	两个请求可依次成功，不产生重叠
两个请求预约不同 roomId	分别锁定不同 meeting_room 行	请求可并行执行，互不阻塞
创建预约与管理员切换资源状态竞争	双方围绕同一 meeting_room 状态形成受控竞争	创建方在锁后重新检查资源状态，避免使用过期“可用”信息

6.3.3 事务归属与回滚原则

创建预约的事务由 ReservationService 持有。Controller 不开启业务事务，Mapper 也不决定事务范围。资源锁定、资源规则检查、冲突查询、状态选择和预约插入必须在同一事务内执行；任一环节失败时整个事务回滚。审批场景由 administration 发起，但预约状态修改与 approval_record 写入同样需要形成单一事务一致性单元。

当前创建预约只涉及单个目标会议室的资源锁，不需要处理多会议室锁顺序。若后续扩展批量预约或一次操作锁定多个会议室，应在新的设计中增加固定锁顺序，避免多资源竞争导致死锁；该要求属于后续扩展约束，不改变当前单会议室预约模型。

6.4 幂等设计

6.4.1 幂等目标

网络重试、用户双击按钮或页面重复提交可能使同一预约意图多次到达后端。如果仅依赖前端按钮禁用，仍无法覆盖网络层重试和并发重复请求。因此系统使用 `requestId` 作为业务幂等键，并在前端、服务层和数据库层形成三层防护。

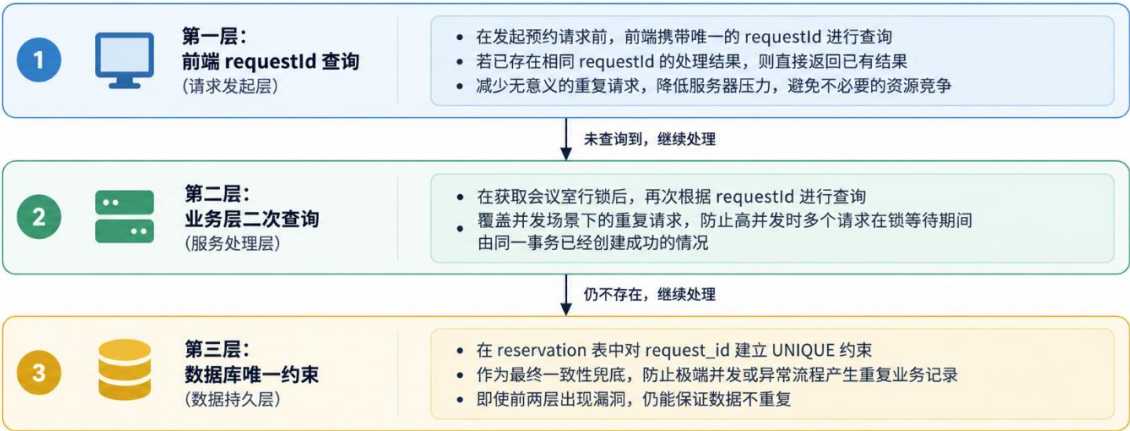


图 6-3 requestId 三层幂等防护机制

表 6-5 幂等控制层次

层次	实现方式	作用
前端生成	每次“新建预约意图”使用 <code>crypto.randomUUID()</code> 生成 <code>requestId</code>	保证一次用户意图拥有稳定标识，重试时沿用同一 <code>requestId</code>
服务层锁前检查	进入资源临界区前先按 <code>requestId</code> 查询	快速返回已经完成的重复请求，减少锁竞争
服务层锁后二次检查	获得 <code>meeting_room</code> 行锁后再二次查询 <code>requestId</code>	覆盖另一个相同请求在本请求等待锁期间已经提交的情况
数据库唯一约束	<code>UNIQUE(request_id)</code>	作为最终一致性兜底，阻止重复业务记录落库

6.4.2 幂等命中与语义冲突

当相同 requestId 对应相同业务语义时，服务端返回既有预约结果而不是再次创建记录。若同一 requestId 被用于不同 roomId、时间区间或其他关键业务参数，则属于幂等语义冲突，应按第 5 章接口契约返回 DUPLICATE_REQUEST，避免客户端错误复用 requestId 导致不可预测结果。

6.5 预约状态机

6.5.1 持久化状态集合

reservation.status 只保存具有业务事实意义的 PENDING、CONFIRMED、REJECTED 和 CANCELLED 四种状态。普通会议室在完整校验通过后直接进入 CONFIRMED；特殊分类在 approval_required=true 时进入 PENDING，并从创建成功开始占用时间资源。

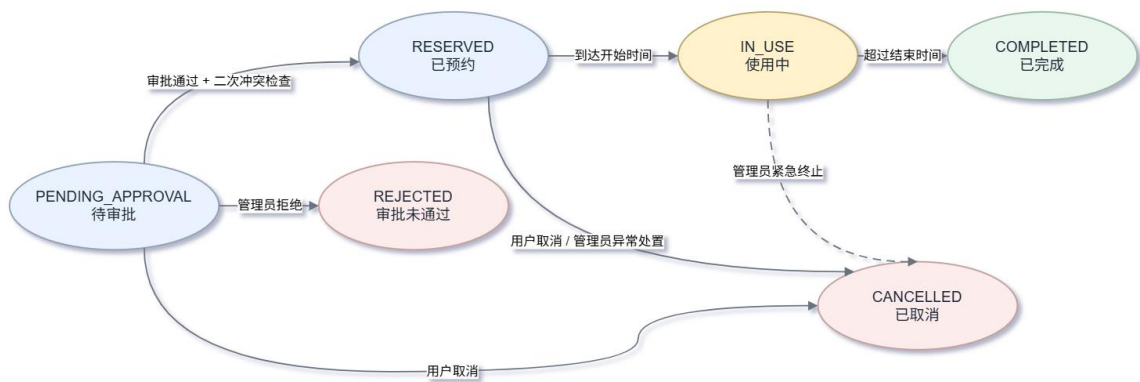


图 6-2 预约持久化状态机

表 6-6 预约持久化状态转换规则

当前状态	触发动作	目标状态	前置条件
新建	创建普通会议室预约	CONFIRMED	资源、时间、容量、 开放规则和冲突检查 全部通过
新建	创建需审批会议室预 约	PENDING	分类 approval_required=true ，其他校验全部通过

当前状态	触发动作	目标状态	前置条件
PENDING	管理员审批通过	CONFIRMED	当前用户为 ADMIN，预约仍处于 PENDING
PENDING	管理员驳回	REJECTED	当前用户为 ADMIN；驳回意见按 设计要求填写
PENDING	用户取消	CANCELLED	本人或 ADMIN，且 $\text{now} < \text{startTime}$
CONFIRMED	用户取消	CANCELLED	本人或 ADMIN，且 $\text{now} < \text{startTime}$

普通会议室在完整校验通过后直接进入 CONFIRMED；特殊分类在 approval_required=true 时进入 PENDING，并立即占用时间资源。管理员审批通过执行 PENDING → CONFIRMED，驳回执行 PENDING → REJECTED。取消操作只针对 PENDING 或 CONFIRMED 且尚未开始的预约，变更为 CANCELLED。PENDING 被驳回不得使用 CANCELLED 表示，以保证审批结果与用户取消语义可区分。

6.6 展示状态推导

表 6-2 展示状态推导

持久化状态/时间条件	displayStatus
CONFIRMED 且 $\text{now} < \text{startTime}$	UPCOMING
CONFIRMED 且 $\text{startTime} \leq \text{now} < \text{endTime}$	IN_USE
CONFIRMED 且 $\text{now} \geq \text{endTime}$	COMPLETED
PENDING	PENDING
REJECTED	REJECTED
CANCELLED	CANCELLED

这种设计避免后台定时任务不断修改“即将开始/进行中/已完成”等状态，也避免由于定时任务失败造成数据与真实时间不一致。前端可以使用后端返回的 displayStatus 展示，但不得向写接口提交 displayStatus。

6.7 取消预约详细设计

6.7.1 取消条件

取消预约属于预约生命周期写操作。服务端根据预约 id 查询记录后，需要依次校验资源是否存在、当前用户是否具有所有权或 ADMIN 权限、预约状态是否允许取消以及预约是否尚未开始。只有 PENDING 或 CONFIRMED 且 $now < startTime$ 的预约能够进入 CANCELLED。

表 6-8 取消预约校验条件

校验项	通过条件	失败处理
预约存在性	目标 reservation 存在	不存在时返回 RESOURCE_NOT_FOUND / 404
所有权/权限	user_id 等于当前用户，或当前用户为 ADMIN	权限不足时返回 FORBIDDEN / 403
当前状态	PENDING 或 CONFIRMED	其他状态返回 RESERVATION_INVALID_STATE / 409
时间条件	$now < startTime$	预约已开始后禁止普通取消

6.7.2 取消结果与审计

取消成功后将 reservation.status 更新为 CANCELLED，并记录 cancel_reason。系统不物理删除预约记录，因为历史预约、取消原因和后续审计均需要保留业务事实。ADMIN 执行治理性取消时，应由 administration 配合写入 operation_log；普通用户正常取消不要求把每一次 HTTP 请求都写入业务审计日志 6.8 特殊会议室审批详细设计（规划）

6.8 特殊会议室审批详细设计（规划）

6.8.1 审批触发条件

审批仅作用于 room_category.approval_required=true 的会议室。此类预约通过创建校验后进入 PENDING，并立即加入同一会议室的时间冲突集合，因此待审批预约已经占用时间资源。该设计避免管理员审批期间出现多个待审申请同时占据同一时间段的空窗。

6.8.2 审批事务

ApprovalService 接收 reservationId、action 和 remark，首先校验当前用户 ADMIN 权限并读取目标预约，随后确认 reservation.status 仍为 PENDING。action=APPROVE 时调用预约生命周期服务执行 PENDING → CONFIRMED；action=REJECT 时执行 PENDING → REJECTED。状态修改与 approval_record 写入应位于同一事务中，避免出现预约状态已改变而审批历史缺失，或审批记录存在但预约状态未改变的情况。

表 6-9 特殊会议室审批规则

审批动作	前置条件	状态变化	记录要求
APPROVE	ADMIN；目标预约为 PENDING	PENDING → CONFIRMED	写入 approval_record，保存 approver_id 与审批意见
REJECT	ADMIN；目标预约为 PENDING	PENDING → REJECTED	写入 approval_record；驳回时 remark 非空
重复审批	目标预约已非 PENDING	不允许再次转换	返回 RESERVATION_INVALID STATE

6.9 前端冲突反馈与刷新

6.9.1 服务端裁决优先

前端展示的空闲状态只用于帮助用户选择时间，不能保证提交时仍然可用。用户在打开日历到点击“提交预约”的间隔内，其他用户可能已经完成预约，因此最终冲突判断必须由服务端事务完成。前端收到冲突响应后，不应本地假设预约成功，也不应通过修改缓存绕过服务器结果。

6.9.2 局部刷新策略

当创建预约返回 409 RESERVATION_TIME_CONFLICT 时，前端保留 title、participantCount、remark 等已经输入的内容，只提示“该时段已被其他用户占用”，并重新请求当前日期和会议室的 calendar 数据。用户在同一表单中重新选择时间或会议室即可，无需整页刷新。该策略同时保证了交互连续性和页面状态与服务器事实重新对齐。

表 6-10 核心写操作的前端反馈策略

服务端结果	前端处理	表单处理
创建成功 201	更新日历与“我的预约”，展示预约结果	清理或关闭新建表单
幂等命中既有结果 RESERVATION_TIME_CONFLICT / 409	按既有预约结果完成成功反馈	不再次产生重复记录
VALIDATION_ERROR	提示时段冲突并局部刷新日历	保留主题、人数、备注等输入
	展示 message 并定位时间/规则问题	保留输入，等待用户修正
UNAUTHORIZED / 401	清理失效会话并引导重新登录	按页面策略保存或放弃临时输入
INTERNAL_ERROR / 500	显示通用失败提示	保留输入，允许用户稍后重试

6.10 本章小结

本章完成了预约核心业务的详细设计。创建预约以 ReservationService 事务为核心，通过会议室行级锁、统一半开区间冲突规则和 requestId 三层幂等防护保证并发一致性；持久化状态机仅保存 PENDING、CONFIRMED、REJECTED 和 CANCELLED 四种业务事实状态，展示状态由当前时间动态推导；取消和特殊会议室审批均通过预约生命周期能力完成，并保留必要的历史与审计信息。

上述设计共同保证了系统最关键的不变量：同一会议室不会因并发请求产生时间重叠的有效预约，同一预约意图不会因重复提交产生多条业务记录，跨模块业务动作也不会绕过预约域直接修改预约事实。该实现基础可直接支撑后续并发验收、审批闭环和会议执行模块扩展。

7 安全、异常与可维护性设计

7.1 身份认证与授权

密码使用 BCrypt 摘要存储；服务端通过 Spring Security 和 JWT 实现无状态身份认证。所有权限判断以服务端为最终依据，前端的管理员按钮显隐仅用于用户体验，不能替代后端角色校验。创建预约不接受前端 userId；取消预约必须再次校验预约所有权。

7.2 统一异常处理

业务异常统一使用 `BusinessException` 携带 `ErrorCode` 和 `HTTP` 状态，由 `GlobalExceptionHandler` 转换为 `ApiResponse`。`Controller` 不直接构造不同格式的错误 JSON。参数校验、认证失败、权限不足、资源不存在、冲突和未预期异常具有固定语义，便于前端按 `code` 做精确交互。

7.3 可维护性约束

- `Controller` 只能调用本域 `Service`，不直接调用 `Mapper`；跨域写操作调用数据所有者 `Service`。
- 业务枚举在前端类型、后端枚举、数据库存储和接口文档中保持一致。
- 数据库变更通过独立 `migration` 进行，不在应用启动时隐式修改生产 `Schema`。
- `Mock` 与真实 `HTTP` 通过 `API adapter` 隔离，组件不感知当前数据源。
- 新增功能优先复用 `ReservationService` 的时间区间和冲突语义，禁止在不同模块复制一套冲突规则。

8 部署与验证设计

8.1 开发运行环境

表 8-1 开发运行环境

组件	端口/版本	说明
Vite 前端	5173	开发服务器，将 <code>/api</code> 代理到 8080
Spring Boot	8080	REST API 服务
MySQL	3306 / 8.x	meeting_room 数据库，InnoDB
Java	21	后端编译与运行基线
Node.js	支持 Vite 6 的版本	前端依赖安装与构建

8.2 构建与测试

前端：

```
cd frontend
npm install
npm run type-check
npm run build
```

```
后端:
cd backend
mvn test
mvn spring-boot:run
```

```
数据库:
mysql -u<user> -p < sql/schema.sql
mysql -u<user> -p < sql/data.sql
```

8.3 并发验收设计

仓库提供 scripts/concurrency-test.py，可对同一 roomId、同一时间段发送大量不同 requestId 的并发创建请求。验收时应满足：至多一个有效预约成功，其余请求返回 409 冲突；测试结束后再次查询日历，数据库中不存在同一会议室相互重叠的 PENDING/CONFIRMED 预约。该测试直接验证系统最关键的并发不变量。

9 实现边界与迭代计划

9.1 当前基线已经完成的设计落地

- 前后端真实 API 已打通，Mock 仅作为可选演示数据源；前端 shared/api 已实现 DTO 适配。
- 后端 identity、resource 查询、reservation 核心链路已具备可运行代码。
- 创建预约已经实现会议室行锁、半开区间冲突、requestId 幂等、统一 409 业务错误。
- MySQL v1.1 Schema 已包含 8 张业务表、外键、CHECK、唯一键和核心索引。

9.2 下一阶段编码优先级

表 9-1 后续迭代优先级

优先级	工作项	完成标准
P0	会议室管理 CRUD 与状态治理	管理员可增改停用，资源规则由 resource 域统一维护

优先级	工作项	完成标准
P0	审批闭环	特殊会议室 PENDING 可 APPROVE/REJECT，并写 approval_record
P1	可用会议室/空闲时段	复用同一半开区间与有效状态集合计算
P1	管理员全量预约与强制取消	所有权/角色校验、operation_log 审计
P1	会议执行能力	参会/签到等不越权修改 reservation
P2	统计看板	预约量、使用率、热门会议室等只读指标

9.3 设计变更控制

若后续需要修改持久化状态集合、时间区间语义、审批占用规则或跨域所有权，应同时更新核心业务契约、API Contract、数据库 migration、前端类型和测试；这类变更不得由单个成员仅在自己的模块中局部修改。普通 UI 调整或只读统计可以在不改变业务契约的前提下独立演进。

10 结论

本系统被设计为“有限共享资源的时段分配与一致性控制系统”。总体采用 Vue 3 + Spring Boot + MyBatis + MySQL 的前后端分离模块化单体架构，以 identity、resource、reservation、meeting、administration 五个业务域组织代码，以 8 张业务表保存资源、预约和审计事实。核心预约链通过会议室行级锁、半开区间冲突判定、requestId 幂等和状态机共同保证并发条件下的数据一致性。该设计在课程项目规模下保持实现复杂度可控，同时为五名成员并行开发和后续审批、会议执行、统计扩展保留明确边界。

附录 A 关键数据与接口约定

A.1 预约与资源状态枚举

表 A-1 状态枚举

类型	取值	含义
用户角色	USER / ADMIN	普通用户 / 管理员
会议室资源状态	AVAILABLE / MAINTENANCE / DISABLED	可预约 / 维护中 / 停用
预约持久化状态	PENDING / CONFIRMED / REJECTED / CANCELLED	待审批 / 已确认 / 已驳回 / 已取消
展示状态	UPCOMING / IN_USE / COMPLETED	由 CONFIRMED 与当前时间动态推导
审批动作	APPROVE / REJECT	通过 / 驳回

A.2 HTTP 状态约定

表 A-2 HTTP 状态语义

HTTP	语义
200	查询、登录、幂等命中既有预约、取消成功
201	首次创建预约成功
400	参数、时间、开放规则、容量等校验失败
401	缺少/无效 JWT 或登录失败
403	角色或资源所有权不足
404	用户、会议室或预约不存在
409	时间冲突、资源状态竞争、非法状态或幂等语义冲突
500	未预期服务器错误

A.3 代码目录与设计映射

表 A-3 代码目录与设计映射

代码路径	对应设计
------	------

代码路径	对应设计
frontend/src/modules/*	前端按五个业务域划分的模块目录
frontend/src/shared/api/*	统一 HTTP、JWT、DTO 映射和 API adapter
backend/src/main/java/com/timeslot/identity	身份认证域
backend/src/main/java/com/timeslot/resource	会议室资源域
backend/src/main/java/com/timeslot/reservation	预约核心域
backend/src/main/java/com/timeslot/meeting	会议执行扩展域
backend/src/main/java/com/timeslot/administration	管理治理与审计扩展域
backend/src/main/java/com/timeslot/common	统一响应、异常与安全基础设施
sql/schema.sql	MySQL v1.1 完整数据库结构
docs/development/*	业务契约、Domain 边界、API Contract、数据库演进规范